PROJECT REPORT

NUMPY DATA FILTERING

1. Introduction

Data filtering is an important step in data analysis. It helps identify relevant records according to specific conditions. NumPy is a Python library that provides efficient tools for working with numerical arrays.

This project demonstrates filtering numerical values using Boolean conditions and Boolean indexing.

2. Objective

The objective is to create a NumPy array and identify values that are above, below, or equal to a selected threshold. The project also demonstrates comparisons using different thresholds.

3. Problem Statement

When working with numerical data, it is often necessary to identify values that satisfy a particular condition. For example, a teacher may want to identify students who scored above 50 marks.

This project uses NumPy to perform such filtering operations without manually checking each value.

4. Tools and Technologies

- Python 3 for programming.
- NumPy for numerical arrays and filtering.
- Visual Studio Code as the development environment.
- GitHub for source-code management and project sharing.

5. Dataset Description

The program uses a sample dataset representing student marks out of 100.

Dataset: [35, 48, 50, 56, 62, 70, 75, 82, 90, 95]

The dataset contains 10 integer values stored in a one-dimensional NumPy array.

6. Methodology

First, the program creates the numerical array using `numpy.array()`.

A threshold is then accepted from the user. If the user presses Enter without entering a value, the default threshold is 50.

The program applies Boolean conditions using comparison operators:

- `>` identifies values greater than the threshold.
- `<` identifies values less than the threshold.
- `==` identifies values equal to the threshold.
- `>=` identifies values greater than or equal to the threshold.
- `<=` identifies values less than or equal to the threshold.

Boolean indexing extracts the elements that satisfy each condition. The program also demonstrates filtering using thresholds of 40, 50, 70, and 80.

7. Results and Analysis

For a threshold of 50, the program produces the following results:

- Values above 50: [56, 62, 70, 75, 82, 90, 95]
- Values below 50: [35, 48]
- Values equal to 50: [50]
- Values greater than or equal to 50: [50, 56, 62, 70, 75, 82, 90, 95]
- Values less than or equal to 50: [35, 48, 50]

These results demonstrate that NumPy correctly selects values according to the specified conditions.

8. Boolean Conditions and Boolean Indexing

A Boolean condition produces True or False for each element in an array.

For example, `data > 50` creates a Boolean array indicating which marks exceed 50.

The expression `data[data > 50]` uses this Boolean array as a filter and returns only the matching values.

This technique makes numerical data selection concise and convenient.

9. Applications

NumPy filtering can be used in student performance analysis, sales analysis, financial data processing, quality control, scientific research, and machine learning preprocessing.

For example, a business can filter sales values above a target or identify measurements that fall outside an acceptable range.

10. Conclusion

The NumPy Data Filtering project demonstrates how Boolean conditions and Boolean indexing can be used to extract specific values from numerical datasets.

It provides practical experience with NumPy arrays, comparison operators,

thresholds, and numerical filtering. These concepts form a useful foundation for more advanced data analysis and machine learning projects.

12. Project Links

GitHub Repository: https://github.com/venky1318/numpy-data-filtering